IE 643: Deep Learning - Theory and Practice **July-Dec 2026**

Project Report: Acoustic Visual Guided Diffusion for Missing View Image Generation

Team Name: Natural Stupidity *Team Members: 24b4502, 24b2191*

Siddhesh Phadke (24b4502) Kushagra Kushwaha (24b2191)

Abstract

A camera often cannot see part of a scene because of occlusion or a limited field of view, while a microphone still hears what is there. We study whether sound can guide a diffusion model to fill such regions with the right content instead of a generic guess. Following advice from our TA, we first solve audio-conditioned *inpainting*: given an image, a mask of the missing region and a sound clip, generate the missing content. We keep a pre-trained Stable Diffusion 1.5 inpainting model frozen and train only a small adapter (5.8M parameters) that turns LAION-CLAP audio features into eight "audio words" placed in the prompt *"a photo of a . . . "*, in the style of AudioToken. The adapter is trained on AVSBench single-source (S4) clips with a mask-weighted denoising loss. On validation data the matched sound gives a lower denoising error than no sound or another clip's sound. On the 739 S4 test clips, CLIP recognises the sounding object in the generated region 88.0% of the time with the clip's audio, against 31.1% for plain inpainting and 86.5% for an oracle text prompt that names the class; the audio also gives the best perceptual similarity (LPIPS 0.423 vs. 0.474). We then extend the model to an unseen view: monocular depth (Depth Anything V2) and a MegaScenes-style camera move reveal the regions the photo never saw, and the same audio model fills them. Over 240 generated views, the sounding object is recognised in the revealed region 60% of the time with audio versus 6% without, rising to 83% for a 20° orbit. Spatial-audio (FOA) conditioning for true missing-view synthesis is our plan for the Novelty Assessment.

1 Introduction

Cameras and microphones fail in different ways. A camera only records what is inside its field of view and in line of sight; a person around a corner, a dog behind a sofa or a vehicle in the dark is simply absent from the image. A microphone records sound from all directions, so the same scene can contain clear acoustic evidence of objects the camera cannot see. Our allotted problem asks for a multimodal diffusion model that uses an observed RGB image together with acoustic information to reconstruct a missing region or an unseen view, and that lets the sound decide *what* is generated rather than hallucinating from the RGB context alone.

This matters wherever a system must reason about what is just out of sight: mobile robots and assistive devices that hear a person before seeing them, surveillance and search-and-rescue under occlusion or low light, and content creation where a soundtrack should drive the visual edit. Plain image inpainting and novel-view generators complete hidden regions from visual context only, so they tend to paint background or the most common object for that context. The question we ask is simple: does conditioning on sound change the generated content in the expected way?

Deep generative models are the natural tool. Latent diffusion models such as Stable Diffusion [12] already generate realistic, context-consistent fill-ins and accept a conditioning signal through cross-attention. Contrastive encoders such as CLAP [20] map audio to semantic embeddings. The missing piece is a learned bridge between the audio embedding and the conditioning space of the image generator, which is what we train.

With our TA we split the problem into two parts (Figure 1). **Approach 1** (this report) is audio-conditioned inpainting with mono audio: it is well posed, has paired data with object masks (AVSBench [24]) and teaches us conditional diffusion. On top of it we built a **look-around** extension that moves the camera with monocular depth and fills the newly revealed region with the same audio model. **Approach 2**, missing-view synthesis guided by spatial (first-order ambisonics, FOA) audio, is our plan for the Novelty Assessment.

The rest of the report is organised as follows. Section 2 summarises our workflow. We survey related literature in Section 3. Our approach, split into the work done before and after the prep-presentation review, is described in Section 4, and the data in Section 5. We give details on experiments in Section 6 and results in Section 7. The plan for the Novelty Assessment is in Section 8, and we conclude in Section 9.

2 Project Workflow

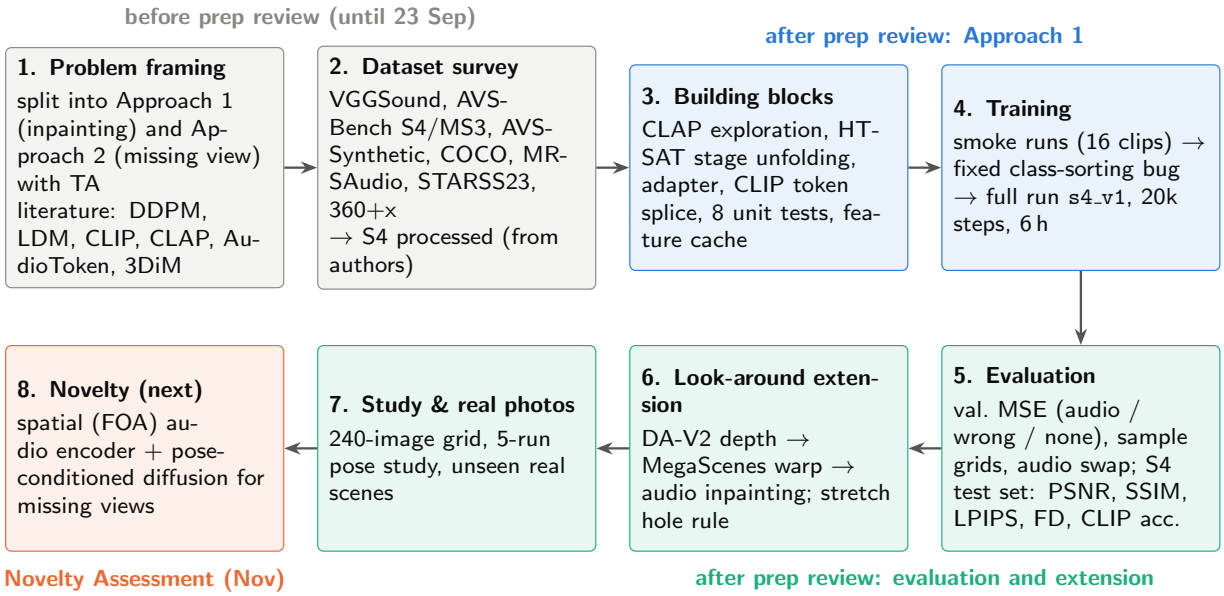


Figure 1: Project workflow. Gray: before the prep-presentation review (23 September 2026). Blue and green: after the review. Orange: the Novelty Assessment.

- **Problem framing (before review).** Read the problem statement and agreed with our TA to first solve mono-audio inpainting (Approach 1) and keep spatial-audio missing-view synthesis (Approach 2) for later. Studied DDPM, latent diffusion, CLIP/CLAP, AudioToken and 3DiM.
- **Dataset survey (before review).** Compared VGGSound, AVSBench S4/MS3, AVS-Synthetic, MS-COCO, MRSAudio, STARSS23 and 360+x. Obtained the processed AVSBench S4 data by writing to the AVSBench authors, and chose it for Approach 1.
- **Building blocks (after review).** Explored CLAP embeddings and text-prompted SD inpainting in notebooks, then wrote a CLAP feature extractor with access to intermediate layers, the audio adapter, the prompt-splicing conditioner and an S4 dataloader, with unit tests for each.
- **Training.** Ran short smoke tests on 16 clips, found and fixed a data-ordering bug, then trained the adapter for 20k steps (run `s4_v1`).

- **Evaluation.** Tracked a validation loss under three conditionings (own, wrong and no audio), inspected sample grids, did audio-swap tests and evaluated the full S4 test split with image-similarity, perceptual and object-consistency metrics against generation baselines.
- **Look-around extension.** Combined Depth Anything V2 depth, MegaScenes camera poses and mesh rendering with our audio inpainting model to fill regions a single photo never saw, and evaluated it on 240 generated views, a 5-run camera-pose study and unseen real photographs.
- **Next: Novelty Assessment.** Spatial-audio (FOA) encoder and pose-conditioned diffusion.

Both team members worked on every stage; we divided the work within each step (for example one of us wrote a module while the other wrote its tests, or one ran experiments while the other inspected and summarised the outputs) rather than splitting the project into separate halves.

3 Literature Survey

Denoising diffusion. Ho et al. [5] define a forward process that gradually adds Gaussian noise to an image, $q(x_t \mid x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$, and train a network $\epsilon_\theta(x_t, t)$ to predict the added noise with the simple loss $\mathbb{E}\,\|\epsilon - \epsilon_\theta(x_t, t)\|^2$. Sampling runs the learned reverse process from pure noise. Unlike a regression model, a diffusion model can represent many plausible completions of an ambiguous input, which is exactly the situation for a hidden region. DDIM [15] gives a deterministic sampler that needs far fewer steps; we use it for all generation.

Latent diffusion and Stable Diffusion inpainting. Rombach et al. [12] run diffusion in the latent space of a pre-trained autoencoder ($512 \times 512 \times 3$ images become $64 \times 64 \times 4$ latents), which makes training and sampling affordable, and condition a U-Net through cross-attention on the output of a text encoder. The Stable Diffusion 1.5 inpainting model [13] extends the U-Net input to 9 channels: the noisy latent, the downsampled mask and the latent of the masked image. Classifier-free guidance [6] mixes conditional and unconditional predictions, $\hat{\epsilon} = \epsilon_\varnothing + s(\epsilon_c - \epsilon_\varnothing)$, to strengthen the conditioning. We keep this whole model frozen and only change what goes into cross-attention.

Contrastive audio and image–text encoders. CLIP [11] aligns images and text with a contrastive loss over 400M pairs; its text encoder is the conditioning encoder of Stable Diffusion. CLAP [3, 20] applies the same idea to audio and text. We use the LAION-CLAP model of Wu et al. [20], whose audio branch is an HTS-AT Swin transformer [1] on log-mel spectrograms, because its embeddings describe *what* a sound is in language-aligned terms. CLIP and CLAP live in different spaces, so an adapter is needed.

AudioToken. Yariv et al. [22] adapt a frozen text-to-image diffusion model to audio by learning a small embedder that maps BEATs audio features into a pseudo-word placed after "A photo of a" in the prompt. Only the embedder is trained, with the usual latent-diffusion loss plus an ℓ_1 penalty on the token and an optional classification loss. This is the closest work to ours: we follow its overall recipe but use CLAP features, produce eight tokens instead of one, and target *inpainting* rather than whole-image generation, so that sound fills a hole while the rest of the scene constrains it. Related work on sound-driven image generation and editing includes Sound2Scene [16] and SoundBrush [17].

Novel-view synthesis. 3DiM [19] treats novel-view synthesis as pose-conditioned image-to-image diffusion; Zero-1-to-3 [10] and later models add camera conditioning to large pre-trained diffusion models. MegaScenes [18] trains scene-level novel-view diffusion and, for single-image inference, warps the input to the target pose using monocular depth, then generates the rest. We reuse its pose schedule and its depth-mesh rendering code, but replace its generator with our audio-conditioned inpainting model. For depth we use Depth Anything V2 [21], a ViT model that predicts relative (affine-invariant) disparity from a single image.

Audio-visual data. AVSBench [24] provides YouTube clips with pixel-level masks of the *sounding* object, which gives exactly the (image, mask, audio) triplets that Approach 1 needs. For spatial audio, STARSS23 [14] and MRSAudio [4] pair FOA recordings with video and source positions; these are the candidate datasets for Approach 2.

4 Proposed Approach

4.1 Work done before the prep-presentation review

Before the review (23 September 2026) our work was planning and survey; no model code had been written.

- **Problem split.** Approach 1: input a partially observed RGB image, a missing-region mask and the corresponding audio; output the completed image. Approach 2: input an RGB view and spatial audio; output a different view of the same scene. Our TA advised mel-spectrogram based (mono) audio and Approach 1 first.
- **Literature.** DDPM, latent diffusion, CLIP and CLAP, AudioToken for audio-to-image adaptation, 3DiM and the newer single-image novel-view models (Zero-1-to-3, Free3D, MultiDiff, Consistent-1-to-3, SV3D, CAT3D) for Approach 2.
- **Proposed architecture.** Audio $\rightarrow$ CLAP $\rightarrow$ audio-to-CLIP adapter $\rightarrow$ cross-attention of a frozen SD-1.5 inpainting U-Net; masked image and mask enter through the VAE as usual.
- **Proposed training.** Two stages: (1) align the adapter with CLIP text space using paired audio–caption data, (2) fine-tune with the inpainting objective, compared against training directly with the inpainting objective.
- **Datasets.** Survey summarised in Section 5; AVSBench S4 (processed frames, masks and audio obtained from the authors) selected for Approach 1, MRSAudio as the main candidate for Approach 2.

4.2 Work done after the prep-presentation review

After the review we implemented, trained and evaluated Approach 1 and built the look-around extension. Two changes from the prep plan are worth stating up front. First, we trained the adapter **directly with the inpainting objective** and skipped the separate audio–caption alignment stage: S4 has class labels but no captions, and AudioToken shows that the diffusion loss alone is enough to place the audio token in a useful part of CLIP space. We kept a label-based classification loss in the code as a cheap substitute for that alignment, but left it off in the reported run. Second, the prep plan only specified "CLAP $\rightarrow$ adapter $\rightarrow$ CLIP space". Following AudioToken, which reads several encoder layers rather than only the final embedding, our adapter also reads three intermediate CLAP layers that keep time and acoustic detail, and it outputs eight tokens instead of one.

4.2.1 Audio encoder: frozen LAION-CLAP with intermediate layers

We use the LAION-CLAP checkpoint `630k-audioset-best` (HTS-AT tiny audio branch). Audio is resampled to 48 kHz mono; a 5 s S4 clip is repeat-padded to CLAP's 10 s window. HTS-AT computes a 1001×64 log-mel spectrogram and folds 1024 frames into a 256×256 "image" (four time chunks stacked vertically) before its four Swin stages. AudioToken takes layers 4, 8 and 12 of a 12-layer ViT; HTS-AT has four stages instead, so we register forward hooks on the outputs of stage 2 (`layers.1`, 16×16 grid, 384 channels), stage 3 (`layers.2`, 8×8, 768) and the final LayerNorm (`norm`, 8×8, 768), plus the 512-d projected and ℓ_2-normalised

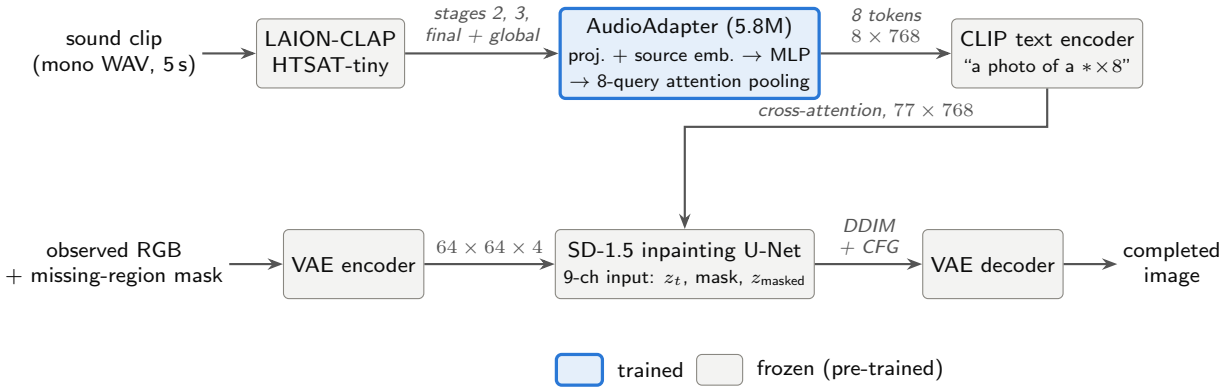


Figure 2: Approach 1. Only the AudioAdapter (blue) is trained. Its eight output tokens replace eight placeholder words of the prompt "a photo of a ∗∗∗∗∗∗∗∗"; the frozen CLIP text encoder turns the prompt into the 77×768 cross-attention context of the frozen SD-1.5 inpainting U-Net.

clip embedding. We undo the frequency–time folding exactly and average over frequency only, which keeps time: the three intermediate sources become sequences of 64, 32 and 32 tokens (about 156 ms and 312 ms per token). A unit test checks that averaging our unfolded final-stage tokens over time reproduces CLAP's own pooled embedding. Mid-level stages keep more acoustic detail; the final stage and global embedding carry the semantic class.

4.2.2 The AudioAdapter (the only trained module)

Let $F_1 \in \mathbb{R}^{64\times384}$, $F_2, F_3 \in \mathbb{R}^{32\times768}$ be the unfolded stage features and $g \in \mathbb{R}^{512}$ the global embedding. The adapter computes

$$
\begin{aligned}
&h_i = F_i W_i + b_i + e_i \quad (i = 1, 2, 3), && h_g = g W_g + b_g + e_g, \\
&H = [h_1;\, h_2;\, h_3;\, h_g] \in \mathbb{R}^{129\times768}, && \tilde{H} = \mathrm{GELU}(H A_1) A_2 \quad (768 \to 1024 \to 768), \\
&T = \mathrm{MHA}(Q,\ \tilde{H},\ \tilde{H}) \in \mathbb{R}^{8\times768}, && Q \in \mathbb{R}^{8\times768} \text{ learned.}
\end{aligned}
$$

- **Per-source projections** W_i, W_g bring each source to the CLIP word width 768 (1.87M parameters). We concatenate sources along the sequence axis rather than the feature axis (as AudioToken does) because the stages have different lengths and widths; this keeps every stage at its native time resolution.
- **Source embeddings** e_i (one learned vector per source) tell the pooling which layer a token came from, since attention pooling is otherwise permutation invariant.
- **Token MLP** A_1, A_2 with GELU is AudioToken's $W_2\,\mathrm{GELU}(W_1 a)$ and maps audio features towards CLIP's word-embedding space (1.57M parameters).
- **Multi-token attention pooling**: eight learned queries attend over the 129 tokens with 8-head attention (2.37M parameters). AudioToken uses one token; we use eight so that the conditioning can describe more than a single word (object, its appearance, its context).

The adapter has 5,817,088 parameters, compared with about 860M in the frozen U-Net.

4.2.3 Injecting audio tokens into Stable Diffusion

The prompt is tokenised as `<bos> a photo of a * * * * * * * * <eos> <pad>...` (77 positions). A temporary forward hook on CLIP's token-embedding layer replaces positions 5–12 with the eight adapter tokens; CLIP then runs unchanged (positional embeddings, causal mask, 12 layers, final LayerNorm) and produces the 77×768 context for the U-Net's cross-attention. We use a hook because the CLIP text model in `transformers` 5 only accepts token ids. CLIP is frozen but not inside `no_grad`, so gradients reach the adapter through it. Because of the causal mask, "a photo of a" never sees the audio while the end and padding positions do, which matches how SD normally reads a prompt. A unit test checks that splicing in the placeholder's own word embeddings reproduces the normal CLIP output exactly, and that gradients reach every adapter parameter while CLIP receives none.

4.2.4 Training objective

For an image x, mask m (1 = missing) and audio a, with frozen VAE encoder $\mathcal{E}$: $z_0 = \mathcal{E}(x)$, $z_m = \mathcal{E}(x \odot (1-m))$, $\bar{m} = m$ resized to 64×64, $t \sim \mathcal{U}\{0, \dots, 999\}$, $\epsilon \sim \mathcal{N}(0, I)$, $z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon$, and context $c = \mathrm{CLIP}(\text{"a photo of a"} \oplus \mathrm{Adapter}(\mathrm{CLAP}(a)))$, replaced by the empty-prompt context with probability 0.1 so that classifier-free guidance works at test time. The loss is

$$\mathcal{L} = \frac{\sum w \odot \big(\epsilon_\theta([z_t, \bar{m}, z_m], t, c) - \epsilon\big)^2}{\sum w} + \lambda_1 \overline{|T|} + \lambda_c \big(1 - \cos(\bar{T}, e_y)\big), \qquad w = 1 + \alpha\,\bar{m}.$$

- **Mask-weighted MSE** ($\alpha = 2$): the usual noise-prediction loss with the missing region weighted three times as much, because the audio can only influence what is generated inside the mask; the rest of the latent is mostly copied from z_m.
- **ℓ_1 token penalty** ($\lambda_1 = 10^{-3}$), from AudioToken, keeps the token norm from growing far beyond that of real CLIP word embeddings.
- **Classification term**: cosine distance between the mean token $\bar{T}$ and the CLIP word embedding e_y of the class label. Implemented, but $\lambda_c = 0$ in the reported run.

Only the adapter's parameters receive gradients; ϵ_θ, $\mathcal{E}$, CLIP and CLAP stay frozen.

4.2.5 Mask mixture

Each training sample uses one of three masks: the dilated ground-truth sounding-object mask (60%), a random box covering 10–50% of the image (20%) or the full image (20%). Object masks teach the main task, drawing the sounding object in context; boxes teach that the sound can also explain arbitrary missing regions; full masks force the tokens to carry enough information to generate the whole scene.

4.2.6 Look-around extension: depth, view warp and audio inpainting

Inpainting a given mask assumes we already know where the hidden region is. To move towards the original "around the corner" scenario we let the *geometry* define the missing region (Figure 3):

1. **Depth.** Depth Anything V2 (ViT-L) predicts relative disparity on the full-resolution photo; it is resized bicubically to 512×512, inverted (disparity clamped at 10^{-3}), divided by its 20th percentile and clipped to $[0, 100]$, as in MegaScenes. Near content then sits at depth ≈ 1, the unit in which MegaScenes expresses camera motion.

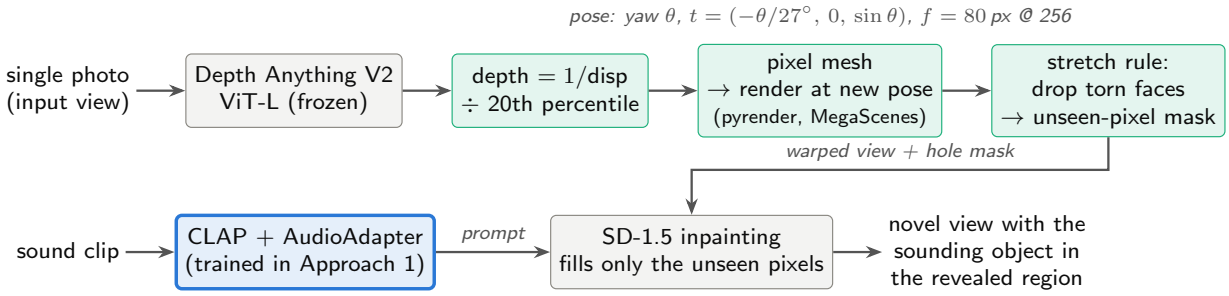


Figure 3: Look-around pipeline. Green: geometry (no learning). Nothing here is trained; the audio model is the Approach 1 checkpoint.

2. **Camera.** Intrinsics $f_x = f_y = 80$ px for a 256 px image (scaled to 512, about 116° field of view), principal point at the centre. The target pose follows MegaScenes' orbit: rotation by yaw θ about the vertical axis and translation $t = (-\theta/27°, 0, \sin\theta)$, i.e. $t = (-0.556, 0, 0.259)$ for $\theta = 15°$. A unit test checks our poses against MegaScenes' own code.
3. **Warp.** Every pixel becomes a mesh vertex (two triangles per pixel quad) that is rendered from the new pose with pyrender using MegaScenes' unmodified mesh and render functions. Pixels that receive no geometry are the holes.
4. **Hole rule (our change).** MegaScenes drops mesh faces seen at more than 70° from the camera. With inverted relative disparity, far and slanted surfaces look very steep, so 19–25% of the input disappears *even with no camera motion*. We instead drop only faces that *tear open* in the new view: a face is removed when its longest edge in the target image is more than 4 times its longest edge in the input image times its zoom factor $\max(z_{\text{src}}/z_{\text{tgt}})$. This happens exactly for faces that bridge a depth discontinuity. Tests verify that the identity pose gives no holes, that the region behind a near object becomes a hole, and that on a continuous plane the holes are exactly the pixels whose rays leave the original frame.
5. **Fill.** The hole mask is dilated by 8 px (the renderer anti-aliases hole edges against black), the audio model inpaints it with 50 DDIM steps and guidance 7.5, and the generated pixels are blended back with a feathered edge so that every pixel the camera saw is kept exactly.

The result is a new view in which only regions the input photo never saw are generated, and the sound decides what appears there.

5 Data set Details

Main dataset: AVSBench single-source (S4) [24]. S4 contains 4,932 YouTube clips of 5 s, each with a single sounding object from one of 23 classes (animals such as dog, cat, horse and lion; vehicles such as bus, race car, helicopter and ambulance; instruments such as guitar, piano, violin, tabla and ukulele; people speaking, singing and a baby laughing; tools such as a chainsaw, lawn mower and keyboard). The data are video frames (RGB images), audio and binary segmentation masks. We received the processed version from the AVSBench authors (it is also listed on the project page `https://opennlplab.github.io/AVSBench/`).

	train	val	test
clips (5 s, 23 classes)	3,452	740	740
RGB frames per clip (224×224, 1 per second)	5	5	5
annotated sounding-object masks per clip	1 (frame 1)	5	5
audio	one WAV per clip, used to condition every frame of the clip		

Table 1: AVSBench S4 as used in this project. Train classes range from 44 (coyote howling) to 217 (helicopter) clips.

Pre-processing.

- **Images**: frames are resized bicubically from 224 to 512 px (SD's native size) and scaled to $[-1, 1]$; random horizontal flips (with the mask) during training.
- **Masks**: binarised at 127, resized with nearest neighbour and dilated by 8 px (max-pool), so the hole also covers the object's boundary and SD cannot copy its outline. Random-box and full-image masks are created on the fly (Section 4).
- **Audio**: resampled to 48 kHz mono, int16 round trip as in LAION-CLAP, repeat-padded to 10 s and cropped deterministically. CLAP features of all 4,932 clips are computed once and cached in fp16 (699 MB, about two minutes), so CLAP never runs during training.
- **Splits**: the official S4 train/val/test splits. Validation uses a class-balanced subset of 64 clips (round robin over classes, frame 1 with its mask); the full 740-clip test split is used only for the final evaluation.

Data used only for qualitative testing. For the look-around experiments we use frame 1 of eight S4 *test* clips as input views and four S4 test clips as sounds (cat, dog, baby, guitar). To test on scenes the model never saw, we also use five frames (restaurant, store, village, forest, classroom) from the Kaggle "Scene classification: images and audio" dataset [7] and one phone photo of a sofa taken by us.

Datasets surveyed but not used. VGGSound [2] (200k clips, but many YouTube links are dead), AVSBench MS3 (424 multi-source clips; left for a multi-object extension), AVS-Synthetic [9] (62k image–audio–mask triplets where image and audio are matched only by class), MS-COCO (no audio). For Approach 2: MRSAudio [4] (FOA and binaural audio with egocentric/exocentric video and 3D trajectories), STARSS23 [14] (FOA, 360° video and source azimuth/elevation/distance) and 360+x. S4 was chosen because image and audio come from the same scene, there is one dominant sounding object, and the sounding-object masks give a principled "missing region".

6 Experiments

Hardware and software. One NVIDIA RTX A5000 (24 GB) for training; two A5000s for evaluation; AMD Ryzen Threadripper PRO 5975WX (32 cores), 125 GB RAM. Python 3.13.5, PyTorch 2.6.0 (CUDA 12.4), diffusers 0.40.0, transformers 5.17.0, laion_clap 1.1.7, pyrender 0.1.45.

6.1 Training the audio adapter

optimizer	AdamW, lr 8×10^{-5}, w. decay 0.01	batch	4×2 (grad. accumulation) $= 8$
schedule	500 warm-up steps, cosine decay	steps	20,000 ($\approx$46 epochs)
gradient clipping	1.0	condition dropout	0.1 (for CFG)
mask-loss weight α	2.0	ℓ_1 weight λ_1	10^{-3}
class-loss weight λ_c	0	mask mix	0.6 object, 0.2 box, 0.2 full
resolution	512 px	seed	0

Table 2: Hyper-parameters of run `s4_v1`. Learning rate and batch size follow AudioToken.

Precision and memory. The frozen U-Net keeps fp32 weights and runs under bf16 autocast with gradient checkpointing; the VAE runs in fp16 under `no_grad`; the CLIP text encoder runs in fp32. Peak GPU memory was 6.0 GB and throughput 0.94 optimizer steps/s, so the 20k steps took about 6 hours. Checkpoints (adapter config + weights + optimizer state) are saved every 2k steps.

Algorithm. Each optimizer step draws two mini-batches of 4 clips, samples a mask type, frame and flip per clip, encodes image and masked image with the VAE (latent sampled), draws t and ϵ, computes the adapter tokens from cached CLAP features, splices them into the prompt, applies condition dropout and evaluates the loss of Section 4.2. Gradients of the two micro-batches are accumulated, clipped to norm 1 and applied with AdamW.

Validation protocol. Every 1k steps we compute the mask-weighted MSE on the 64 validation clips with *fixed* noise and *fixed* timesteps (evenly spaced in $[50, 950]$), so values are comparable across checkpoints. Three conditionings are scored: the clip's own audio, the audio of another clip in the batch (shuffled), and the empty prompt (no audio). The success criterion is own $<$ no audio $<$ shuffled. Every 2k steps we also save a sample grid (DDIM, 30 steps, guidance 7.5; columns: masked input, audio, ground truth, no audio with the same seed).

Smoke tests and a bug they caught. Before the full run we trained on 16 clips for 300–600 steps to check the full loop (training, validation, sampling, checkpointing). The S4 metadata file is sorted by class, so "the first 16 clips" were all helicopters and the shuffled-audio control compared a helicopter sound with another helicopter sound. Subsets are now taken round-robin over classes (`smoke_balanced` run). We also wrote 8 unit tests for shapes, the HTS-AT unfolding, gradient flow (only the adapter receives gradients), batching, save/load, the CLIP splice and U-Net compatibility.

6.2 Test-set evaluation of audio-conditioned inpainting

All 740 S4 test clips are inpainted (frame 1, its ground-truth sounding-object mask dilated by 8 px, 512×512) with the final checkpoint under four conditionings, with the *same seed and mask* per clip so that only the conditioning differs: **own audio** (ours); **wrong audio**, the audio of a test clip from a different class; **no audio**, the empty prompt, i.e. plain SD-1.5 inpainting (the visual-only generation baseline); and **text oracle**, the prompt "a photo of a ⟨class label⟩", an upper bound that assumes the class is known. Sampling uses 30 DDIM steps and guidance 7.5, and generated pixels are pasted back into the original frame. One clip (`tA_NrYVTlps`) has an empty frame-1 mask and is left out of the scores, so all numbers are over 739 clips. Metrics:

- **Image similarity**: PSNR over the masked pixels and SSIM on the mask's bounding box, both against the original frame.
- **Perceptual quality**: LPIPS (AlexNet) [23] on the bounding box resized to 256 px; and FD-CLIP,

the Fréchet distance between CLIP ViT-L/14 features of generated and real region crops (an FID computed with CLIP features [8]).

- **Object consistency / audio–visual alignment**: CLIP ViT-L/14 zero-shot classification over the 23 S4 classes (prompts "a photo of a dog." etc.) of the generated region only (bounding box, pixels outside the mask set to gray). Top-1 accuracy against the clip's class measures whether the sounding object was generated; for wrong audio we also measure accuracy against the *audio's* class.

6.3 Look-around experiments

Main grid (v1). 8 S4 test scenes (desk, doorway with a dog, fireplace room, piano room, bathroom door, bookshelf room, country path, bus street) × 2 poses (±15° MegaScenes orbit) × 5 conditionings (no audio, cat, dog, baby, guitar) × 3 seeds = 240 images, 50 DDIM steps, guidance 7.5. Audio and no-audio images share seeds and masks. Each image is scored with CLIP ViT-L/14 zero-shot over the 23 classes on *only the generated pixels*. A hit means the top-1 class is the sound's class.

Pose study (v2). Same scenes, sounds and seeds 0–1, varying only the camera: no forward move at −15° (MegaScenes' clockwise orbit also moves the camera forward, which zooms in and shrinks the holes), rotation only, a narrower 65° field of view (focal 200 px at 256), and ±20°. We report 95% intervals bootstrapped over scenes and split holes into frame holes (the band rotated into view), interior holes (disocclusions) and "roomy" holes (>32 px from any seen pixel).

Unseen real scenes. Five Kaggle scene photographs and our sofa photo, with the two best poses from the study (+20°, and −15° moving back), four sounds, no-audio baseline, seeds 0 and 1.

Running the model (interface). All experiments are driven by command-line scripts that accept user inputs: `sample_audio_inpaint.py --image --mask --wav` for inpainting any image with any sound (or an S4 clip by id), and `novel_view_audio_inpaint.py --image --wav --yaw-deg` for the look-around pipeline; each writes the outputs and an overview sheet that also shows the no-audio result with the same seed.

7 Results

7.1 Training behaviour

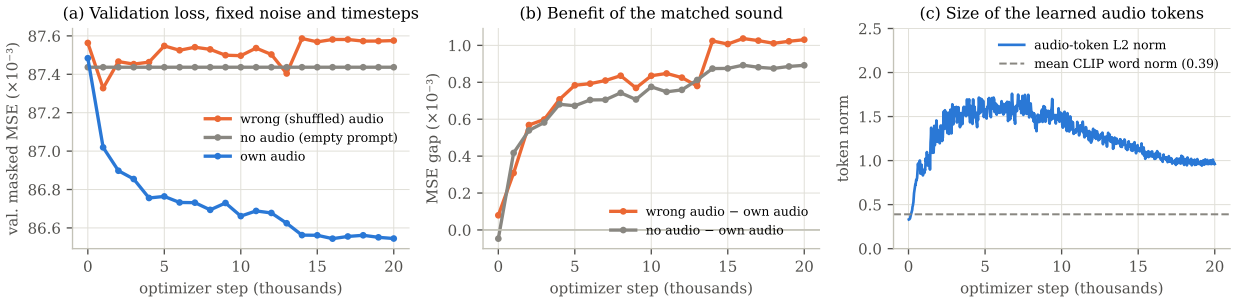


Figure 4: Run `s4_v1`. (a) Validation mask-weighted MSE with fixed noise and timesteps for the clip's own audio, another clip's audio and the empty prompt. (b) Gap between the controls and the own audio. (c) Mean ℓ_2 norm of the audio tokens.

step	own audio	wrong audio	no audio	gap (wrong − own)
0 (untrained)	0.08748	0.08756	0.08744	+0.00008
2,000	0.08690	0.08747	0.08744	+0.00057
10,000	0.08666	0.08750	0.08744	+0.00084
20,000	**0.08654**	0.08758	0.08744	+0.00103

Table 3: Validation MSE (lower is better). The no-audio value does not depend on the adapter.

The matched audio gives the lowest validation loss from the first evaluation onwards and the gap keeps widening until the last step, so there is no sign of over-fitting and the final checkpoint is the best one (Figure 4, Table 3). The order own < none < wrong is the one we wanted: a correct sound helps, and a wrong sound is worse than no sound because it pushes the model towards the wrong object. The absolute gaps look small because most of the denoising error at a random timestep is irreducible noise that no conditioning can explain; the fixed-noise protocol makes even small differences reliable. The token norm rises quickly from 0.33 to about 1.6 and is then pulled back to 0.96 by the ℓ_1 term while the loss keeps improving, i.e. the adapter first finds a direction in CLIP space and then compresses it.

7.2 Audio-conditioned inpainting on the S4 test set

conditioning	PSNR ↑	SSIM ↑	LPIPS ↓	FD-CLIP ↓	CLIP top-1 ↑	P(class) ↑
no audio (SD-1.5, visual only)	**10.97**	0.516	0.474	0.224	31.1%	0.254
wrong audio (other class)	9.94	**0.525**	0.465	0.131	14.3%	0.130
own audio (ours)	9.97	0.519	**0.423**	**0.080**	**88.0%**	**0.839**
text oracle (class name)	10.35	0.525	0.447	0.116	86.5%	0.836

Table 4: Audio-conditioned inpainting on 739 AVSBench S4 test clips (all 23 classes; same seed and mask for every conditioning). PSNR on masked pixels, SSIM and LPIPS on the mask's bounding box, FD-CLIP between generated and real region crops, CLIP top-1 and P(class) for the clip's true class on the generated region. Best non-oracle value in bold. 95% intervals over clips: PSNR ±0.17 dB, LPIPS ±0.008, top-1 ±2.3 points. CLIP top-1 on the real region: 93.1%.

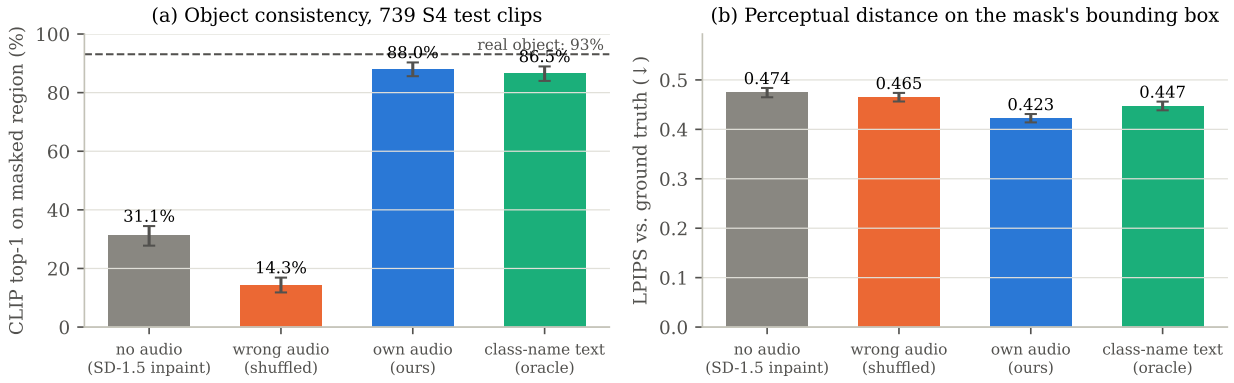


Figure 5: S4 test set. (a) CLIP zero-shot top-1 accuracy of the generated region for the clip's true class (95% intervals over clips); the dashed line is the same classifier on the real region. (b) LPIPS between the generated and the real region.

Table 4 and Figure 5 compare our model with the visual-only baseline (the same SD-1.5 inpainting model with an empty prompt), with a wrong sound, and with an oracle text prompt that names the class.

Object consistency and audio–visual alignment. With the clip's own audio, CLIP recognises the sounding object in the generated region in 88.0% of the 739 test clips, against 31.1% without audio. This matches the oracle text prompt (86.5%), which is told the class label, and comes close to the 93.1% that the same classifier reaches on the real object. For 92.0% of the clips the sound raises the probability of the true class over the no-audio image with the same seed and mask. **The generated content follows the sound, not the image**: with a wrong sound the accuracy for the image's own class falls to 14.3%, below the no-audio baseline, and the region is classified as the class of the *sound* in 48.3% of the clips, even though the surrounding scene belongs to a different class.

Perceptual quality. Our model also gives the generated region closest to the real one in perceptual terms: LPIPS 0.423 against 0.474 without audio (lower for 73.7% of clips), and FD-CLIP 0.080 against 0.224. It is even slightly better than the text oracle (LPIPS 0.447, FD-CLIP 0.116). A likely reason is that the eight audio tokens were trained on S4 frames, so besides the class they also carry what such objects usually look like in these videos (scale, pose, framing), which a three-word class name does not.

Pixel metrics. PSNR is slightly *higher* without audio (10.97 dB vs. 9.97 dB) and SSIM is almost the same for all conditionings. This is expected for generative inpainting of a whole object: with no conditioning SD tends to fill the hole with smooth background whose colours are close to the average of the region, whereas a correctly generated dog or guitar with a different pose or colour from the real one is penalised pixel by pixel. All PSNR values are low (around 10–11 dB) because the masks cover the entire object, which has many plausible appearances. We therefore read PSNR/SSIM as a check that the result stays consistent with the scene, and use LPIPS, FD-CLIP and CLIP accuracy to judge whether the right content was generated.

Per class. The best classes are playing acoustic guitar (100%), mynah bird singing (100%), cat meowing (100%); the weakest are playing ukulele (47%), coyote howling (67%), chainsawing trees (68%) (Figure 6). The weak classes are mostly confusions with a similar-sounding object: 16 of the 30 ukulele clips are drawn as acoustic guitars and 8 of the 37 chainsaw clips as lawn mowers, while coyote howling has only 9 test clips. We have not yet run a formal human evaluation; the qualitative grids below are our own visual inspection, and a small human study is part of our Novelty Assessment plan.

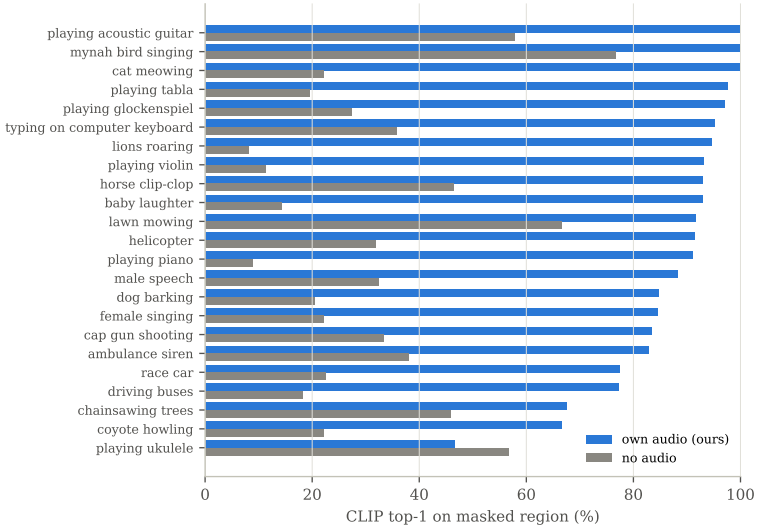


Figure 6: CLIP top-1 accuracy per S4 class on the test set, own audio vs. no audio.

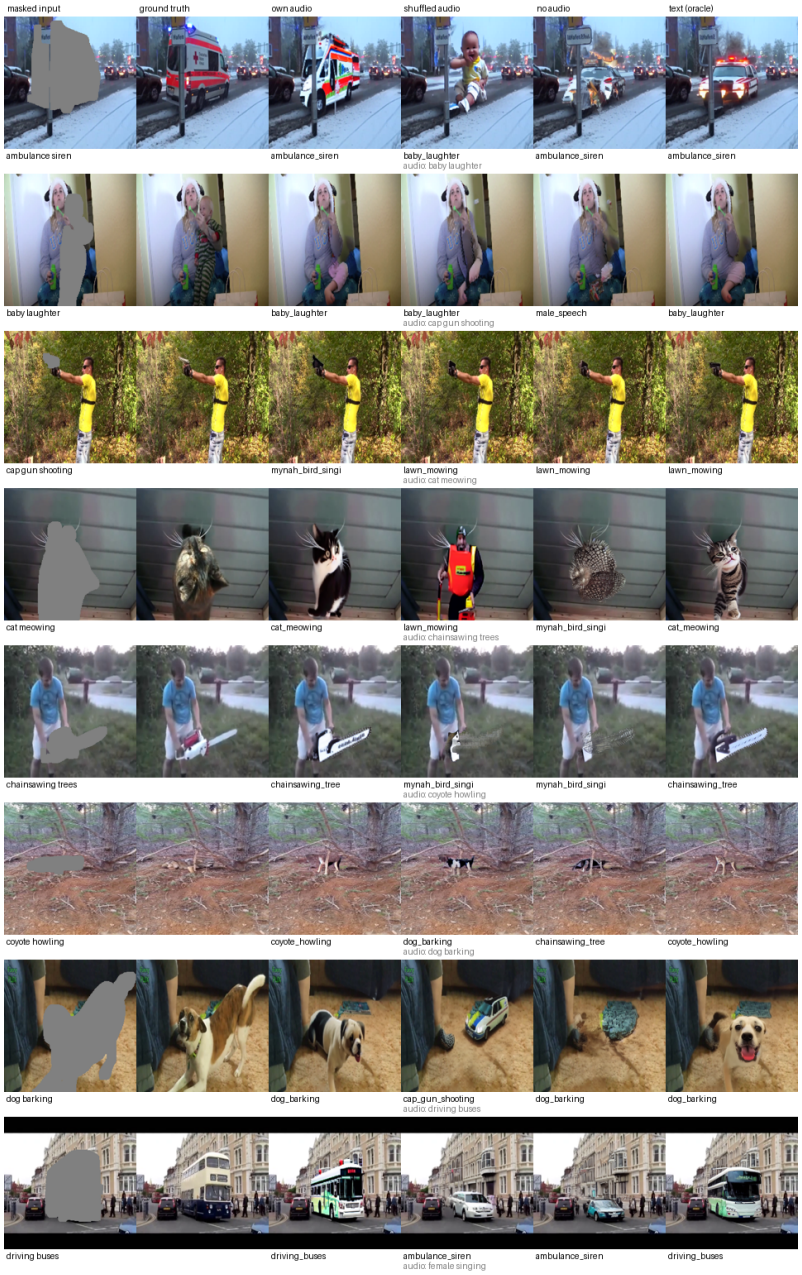


Figure 7: The first eight test clips in class-balanced order (not cherry-picked). Columns: masked input, ground truth, own audio, wrong audio (its class in gray), no audio, text oracle; captions give the CLIP top-1 class of the generated region.

7.3 Qualitative results

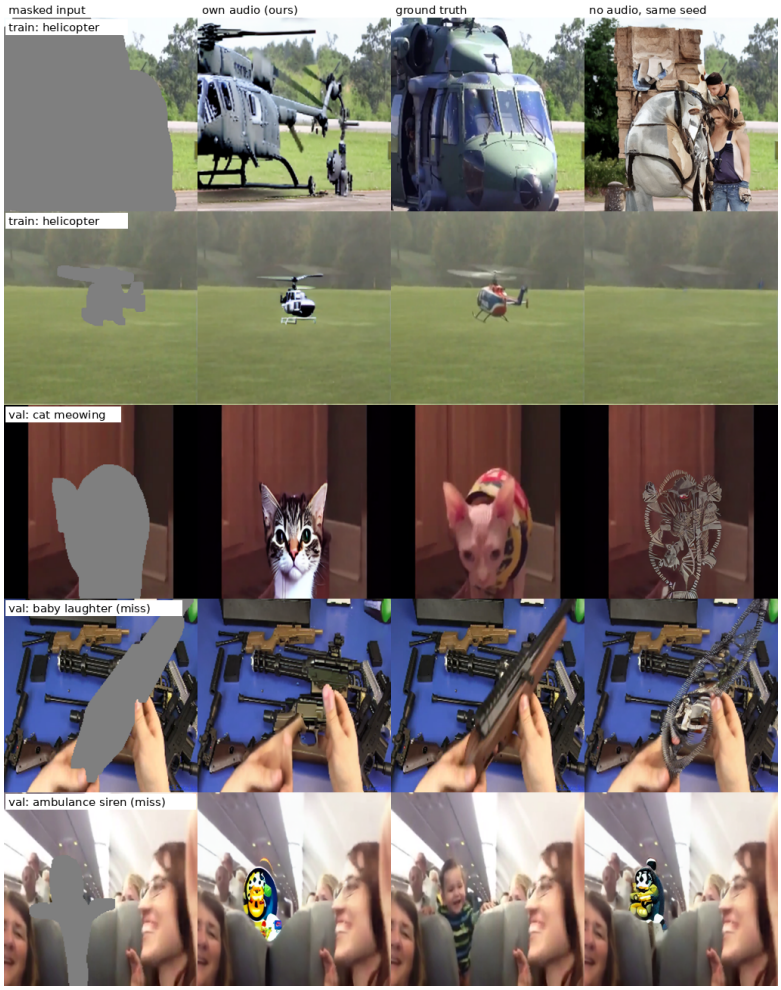


Figure 8: Validation/training samples at step 20k (DDIM 30 steps, guidance 7.5). Columns: masked input, our result with the clip's audio, ground truth, same seed without audio. The last two rows are failure cases: a baby-laughter clip produces a cartoon toy, and a small distant ambulance is not drawn at all.

Figures 8 and 9 show typical behaviour. With the clip's audio, the model draws the sounding object at the right scale and in a pose that fits the mask (helicopters on a runway or in the sky, a kitten, a gun held in a hand), while the no-audio baseline fills the same mask with background, ornaments or unrelated objects. The audio-swap test is the clearest evidence that the generated content follows the sound rather than the visual context: the same masked frame becomes a dog or a cat depending only on the audio. The main failures are classes whose sound is ambiguous or rare in training (baby laughter, 63 training clips) and very small masks, where there are too few pixels for any object. Outputs inherit the blur of the $224 \rightarrow 512$ upscaled frames, and text-like artefacts sometimes appear because many S4 YouTube frames carry captions or logos.

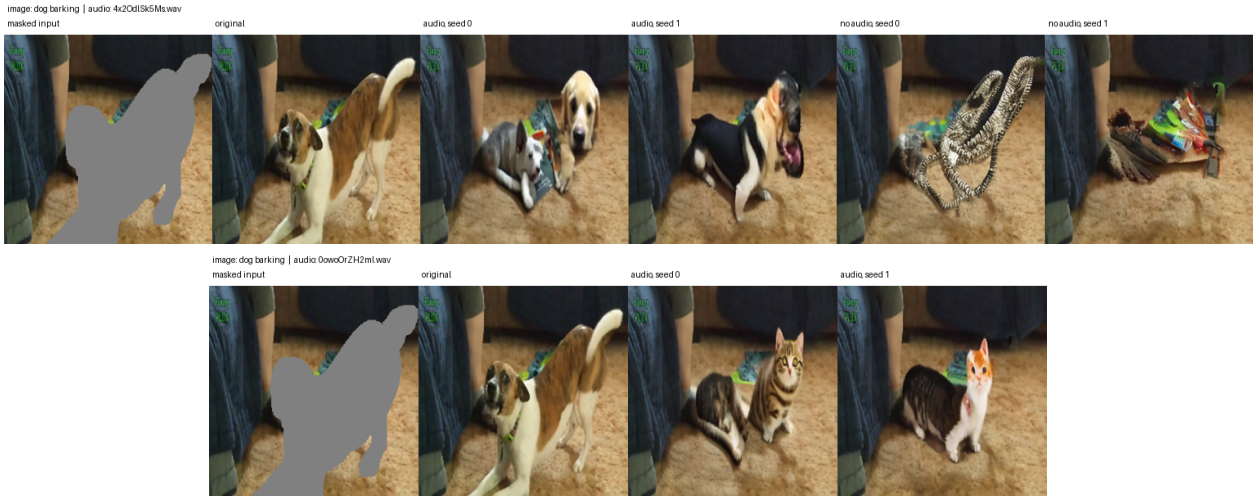


Figure 9: Audio swap on an unseen test clip (`4x2OdlSk5Ms`). Top: with its own dog-barking audio both seeds produce a dog, while the empty prompt with the same seeds produces a coil and clutter. Bottom: same frame, mask and seeds, but cat audio: both seeds produce cats. The sound decides what is drawn.

7.4 Look-around: sound decides what appears in the unseen region

sound	top-1, audio	top-1, no audio	P(class) audio / no audio	paired win
cat	**52%**	4%	0.47 / 0.05	77%
dog	**71%**	12%	0.59 / 0.11	85%
baby	**67%**	6%	0.63 / 0.08	81%
guitar	**52%**	0%	0.38 / 0.01	90%
all	**60%**	6%		83%

Table 5: Look-around grid v1 (8 scenes, ±15°, 3 seeds; 48 images per sound). CLIP sees only the generated pixels. "Without audio" scores the same no-audio images against the row's class; "paired" is the fraction of images where the sound raises the class probability over the no-audio image with the same seed and mask.

Without audio the revealed regions are filled with background, furniture or people; the requested object appears in only 6% of images. With audio it appears in 60% (Table 5), and in 83% of paired comparisons the sound raises the probability of its class. The probability matrix (Figure 10, left) is close to diagonal; the main confusion is cat audio producing dogs (0.15).

The pose study (Figure 10, right) explains most of the variation. At −15° the MegaScenes orbit also moves the camera forward, the view zooms in and the unseen area shrinks to 19%, mostly thin slivers; hit rate 48%. Removing the forward motion raises it by 19 points to 67% (95% CI of the paired difference +0 to +34), and a +20° orbit, which reveals more room (21% of the view farther than 32 px from any seen pixel), reaches 83% (+20 points over +15°). Rotation-only motion and a narrower field of view made no clear difference. Over all 72 scene-views the Spearman correlation between roomy hole area and hit rate is +0.25: the sound wins when the hole has room for a new object.

Figures 11 and 12 show the behaviour on S4 scenes and on photographs the model has never seen. The pipeline generalises to indoor and outdoor scenes outside S4: dogs and cats appear behind the sofa, beside the restaurant tables or on the forest path, and a guitar player appears in the revealed band, while the

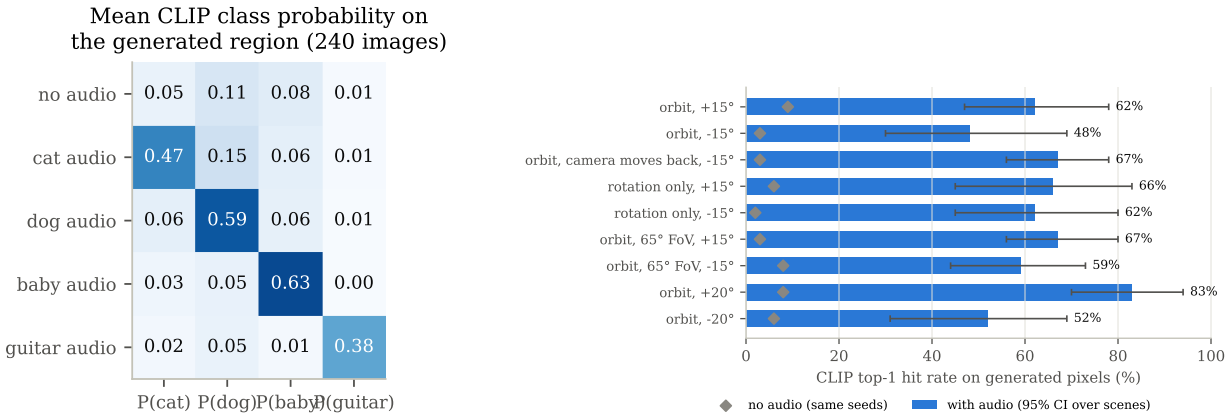


Figure 10: Left: mean CLIP probability of each class (columns) for each conditioning (rows) on the generated region of the 240 look-around images; the diagonal dominates. Right: pose study, audio top-1 hit rate per camera move (64 images per bar, 95% interval over scenes) and the no-audio rate with the same seeds.

no-audio row continues walls, floors and people.

Negative results and failure modes.

- **Thin holes attached to an object** are completed as that object with or without audio (a dog-shaped gap behind a dog extends the dog). Audio only steers regions with room for a new object.
- **MegaScenes' angle-based hole rule** removed 47–57% of each +15° view and shredded walls and floors into streaks, giving the audio model meaningless holes; our stretch rule removes 30–44%, only what was really unseen.
- **Text and watermark artefacts** appear in many large holes, more often with audio. The adapter was trained on YouTube frames, many of which carry overlays, so the tokens partly encode that style.
- **CLIP scoring is conservative**: an object held by a generated person (a guitar played by a woman) is often labelled as "a woman singing"; we treat the metric as a coarse check alongside the contact sheets.

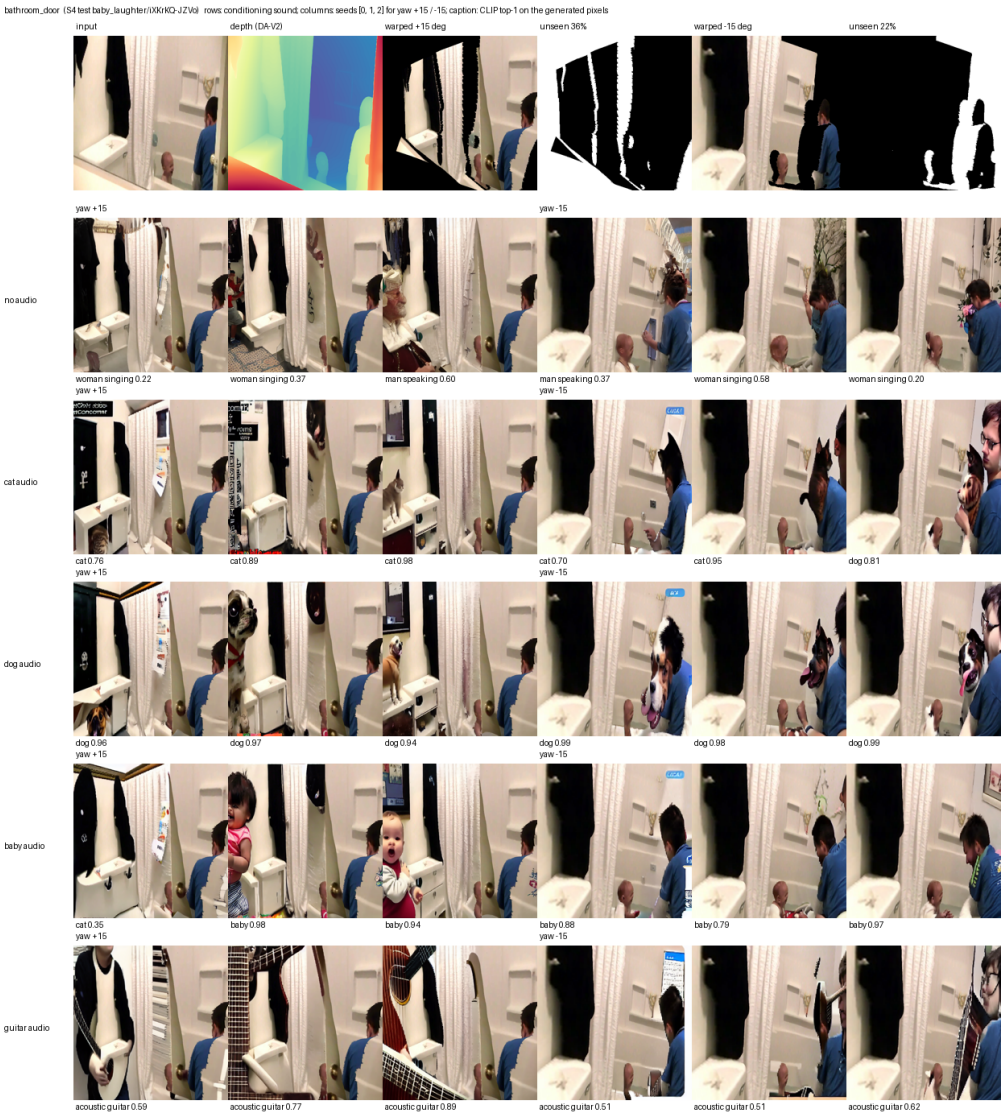


Figure 11: Contact sheet of one look-around scene (bathroom door, S4 test). Header: input, depth, warped views at $\pm 15^\circ$ with black unseen pixels, and the unseen masks. Rows: conditioning sound; columns: three seeds per pose. Captions: CLIP top-1 class and probability on the generated pixels.

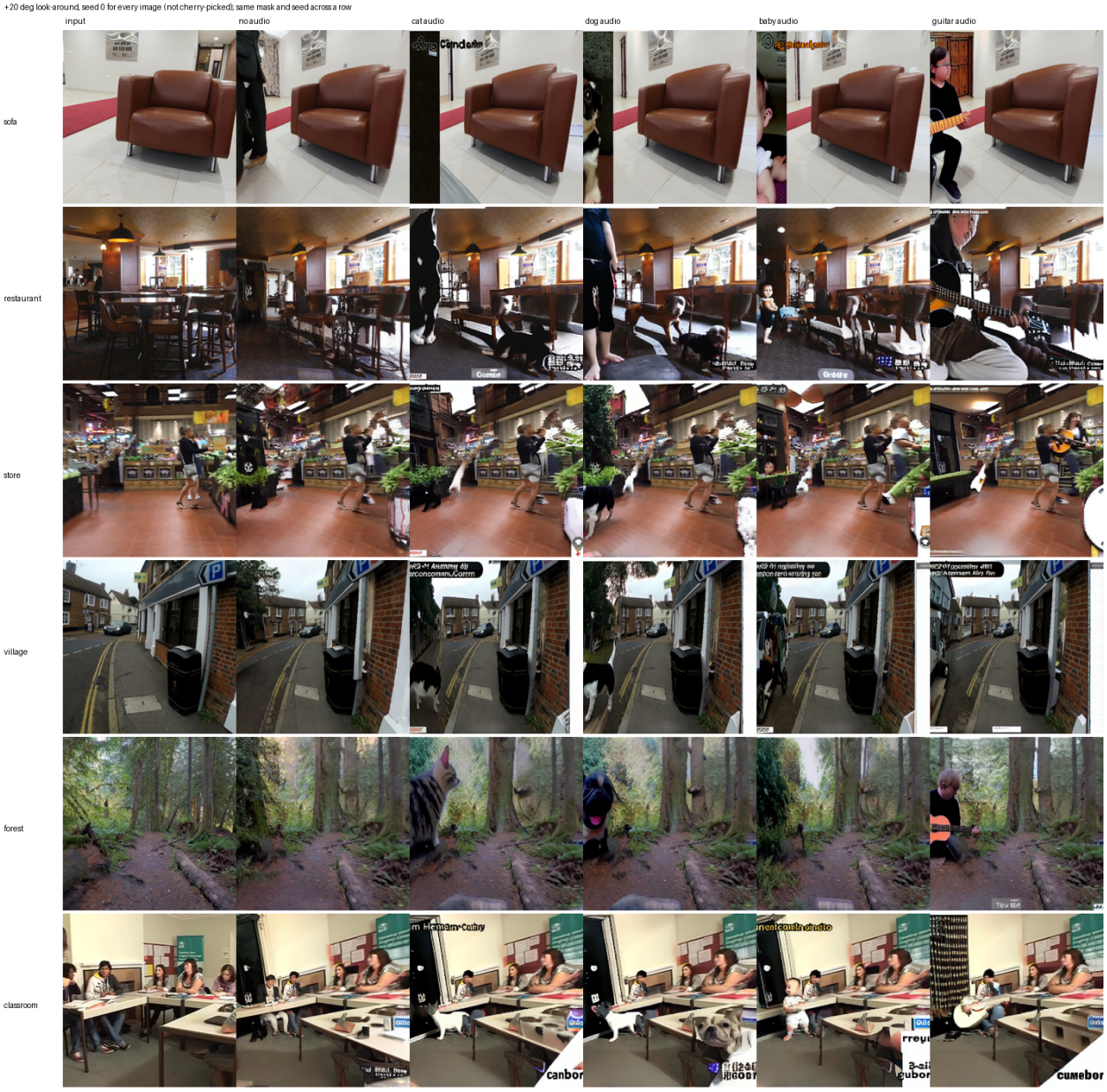


Figure 12: Unseen real scenes (five Kaggle photographs and our own sofa photo) at a +20° orbit, seed 0 for every image (not cherry-picked; same mask and seed across a row). The sound's object appears in about 20 of the 24 audio images and never without audio. Misses: garbled text instead of a cat on the sofa, the guitar in the village, and the baby in the forest and the village.

8 Plan for Novelty Assessment

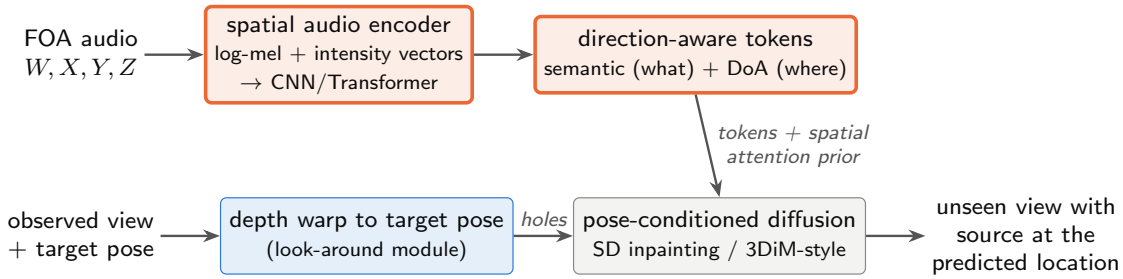


Figure 13: Planned Approach 2: a trained spatial-audio encoder (orange) provides both semantic and direction-of-arrival tokens; the look-around module and the inpainting diffusion model are reused.

For the Novelty Assessment we will implement Approach 2 of our prep presentation: *missing-view synthesis guided by spatial audio*, $I_{\text{obs}} + A_{\text{FOA}} + P_{\text{target}} \rightarrow I_{\text{unseen}}$. Mono audio tells the model *what* is hidden; the four FOA channels (W, X, Y, Z) also tell *where* it is. Our current system can only put the object somewhere in the revealed region, with no control over where; the goal is to place it at the direction the sound comes from.

- **Data.** Curate visible-view / unseen-view pairs from MRSAudio (FOA, egocentric and exocentric video, 3D trajectories), with STARSS23 (FOA, 360° video, source azimuth and elevation) for training the direction branch, and SoundSpaces 2.0 for controlled renderings with known source and receiver geometry; Ego-Exo4D is a candidate for real-world validation. From 360° video we can cut a visible perspective view and a target view with known relative pose and source direction.
- **Spatial audio encoder.** Log-mel spectrograms of the four channels plus acoustic intensity vectors (the standard SELD input) into a CNN/Transformer that outputs semantic tokens (initialised or distilled from our CLAP adapter) and a direction-of-arrival estimate, trained with a localisation loss on STARSS23.
- **Fusion.** Keep the frozen diffusion backbone. Project the source direction into the target view to obtain an attention prior (where in the image the object should be) and add it to the cross-attention of the audio tokens, or condition the inpainting mask on it; compare with a 3DiM-style pose-conditioned model.
- **Experiments.** Visual-only vs. mono audio (our current model) vs. FOA. Metrics: our current ones plus localisation error between the generated object (detected with an open-vocabulary detector) and the true source direction, and a small human study.
- **Timeline.** Data curation and the FOA encoder first (early November), fusion with the frozen diffusion model next, and the comparison experiments in the week before the assessment (23 and 25 November 2026).

9 Conclusion

We addressed the reconstruction of missing image content guided by sound. A 5.8M-parameter adapter maps frozen LAION-CLAP features to eight prompt tokens for a frozen Stable Diffusion 1.5 inpainting model and is trained on AVSBench S4 in six hours on one GPU. The matched sound lowers the validation denoising error below both no sound and a wrong sound, and swapping the sound swaps the generated object. On the

S4 test set the generated region shows the sounding object in 88.0% of clips against 31.1% for visual-only inpainting, matching a text prompt that is told the class (86.5%), and a wrong sound makes the model draw the *sound's* object instead. Combined with monocular depth and a MegaScenes-style camera move, the same model fills regions a single photo never saw: the requested object appears in 60% of revealed regions with audio against 6% without, and in up to 83% with a better camera move, including on real photographs outside the training data. The main limitations are low-resolution training frames, text artefacts inherited from YouTube frames, weak control in thin holes, and the absence of spatial information in mono audio, which our Novelty Assessment will address with FOA audio.

Code. Our code, trained adapter checkpoint and evaluation outputs are available at `https://github.com/siddheshsp0/ie-643-project`. The repository README describes the environment, data layout and the commands to train, evaluate and run the model on new inputs.

References

[1] Ke Chen, Xingjian Du, Bilei Zhu, Zejun Ma, Taylor Berg-Kirkpatrick and Shlomo Dubnov. HTS-AT: A Hierarchical Token-Semantic Audio Transformer for Sound Classification and Detection, IEEE ICASSP, 2022. `https://arxiv.org/abs/2202.00874`

[2] Honglie Chen, Weidi Xie, Andrea Vedaldi and Andrew Zisserman. VGGSound: A Large-Scale Audio-Visual Dataset, IEEE ICASSP, 2020. `https://arxiv.org/abs/2004.14368`

[3] Benjamin Elizalde, Soham Deshmukh, Mahmoud Al Ismail and Huaming Wang. CLAP: Learning Audio Concepts from Natural Language Supervision, IEEE ICASSP, 2023. `https://arxiv.org/abs/2206.04769`

[4] Wenxiang Guo et al. MRSAudio: A Large-Scale Multimodal Recorded Spatial Audio Dataset with Refined Annotations, NeurIPS Datasets and Benchmarks Track, 2025. `https://arxiv.org/abs/2510.10396`

[5] Jonathan Ho, Ajay Jain and Pieter Abbeel. Denoising Diffusion Probabilistic Models, NeurIPS, 2020. `https://arxiv.org/abs/2006.11239`

[6] Jonathan Ho and Tim Salimans. Classifier-Free Diffusion Guidance, NeurIPS Workshop on Deep Generative Models, 2021. `https://arxiv.org/abs/2207.12598`

[7] Jordan J. Bird. Scene Classification: Images and Audio, Kaggle dataset. `https://www.kaggle.com/datasets/birdy654/scene-classification-images-and-audio`. Accessed on: 25th October, 2026.

[8] Tuomas Kynkäänniemi, Tero Karras, Miika Aittala, Timo Aila and Jaakko Lehtinen. The Role of ImageNet Classes in Fréchet Inception Distance, ICLR, 2023. `https://arxiv.org/abs/2203.06026`

[9] Jinxiang Liu et al. Annotation-Free Audio-Visual Segmentation, WACV, 2024. `https://jinxiang-liu.github.io/anno-free-AVS/`

[10] Ruoshi Liu, Rundi Wu, Basile Van Hoorick, Pavel Tokmakov, Sergey Zakharov and Carl Vondrick. Zero-1-to-3: Zero-shot One Image to 3D Object, ICCV, 2023. `https://arxiv.org/abs/2303.11328`

[11] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger and Ilya Sutskever. Learning Transferable Visual Models From Natural Language Supervision, ICML, 2021. `https://arxiv.org/abs/2103.00020`

[12] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser and Björn Ommer. High-Resolution Image Synthesis with Latent Diffusion Models, CVPR, 2022. `https://arxiv.org/abs/2112.10752`

[13] Robin Rombach and Patrick Esser (Runway, Stability AI). Stable Diffusion Inpainting (v1.5 inpainting model card). `https://huggingface.co/stable-diffusion-v1-5/stable-diffusion-inpainting`. Accessed on: 25th October, 2026.

[14] Kazuki Shimada, Archontis Politis, Parthasaarathy Sudarsanam, Daniel Krause, Kengo Uchida, Sharath Adavanne, Aapo Hakala, Yuichiro Koyama, Naoya Takahashi, Shusuke Takahashi, Tuomas Virtanen and Yuki Mitsufuji. STARSS23: An Audio-Visual Dataset of Spatial Recordings of Real Scenes with Spatiotemporal Annotations of Sound Events, NeurIPS Datasets and Benchmarks Track, 2023. https://arxiv.org/abs/2306.09126

[15] Jiaming Song, Chenlin Meng and Stefano Ermon. Denoising Diffusion Implicit Models, ICLR, 2021. https://arxiv.org/abs/2010.02502

[16] Kim Sung-Bin, Arda Senocak, Hyunwoo Ha, Andrew Owens and Tae-Hyun Oh. Sound to Visual Scene Generation by Audio-to-Visual Latent Alignment, CVPR, 2023. https://arxiv.org/abs/2303.17490

[17] Kim Sung-Bin et al. SoundBrush: Sound as a Brush for Visual Scene Editing, AAAI, 2025. https://arxiv.org/abs/2501.00645

[18] Joseph Tung, Gene Chou, Ruojin Cai, Guandao Yang, Kai Zhang, Gordon Wetzstein, Bharath Hariharan and Noah Snavely. MegaScenes: Scene-Level View Synthesis at Scale, ECCV, 2024. https://arxiv.org/abs/2406.11819

[19] Daniel Watson, William Chan, Ricardo Martin-Brualla, Jonathan Ho, Andrea Tagliasacchi and Mohammad Norouzi. Novel View Synthesis with Diffusion Models, ICLR, 2023. https://arxiv.org/abs/2210.04628

[20] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Taylor Berg-Kirkpatrick and Shlomo Dubnov. Large-Scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation, IEEE ICASSP, 2023. https://arxiv.org/abs/2211.06687

[21] Lihe Yang, Bingyi Kang, Zilong Huang, Zhen Zhao, Xiaogang Xu, Jiashi Feng and Hengshuang Zhao. Depth Anything V2, NeurIPS, 2024. https://arxiv.org/abs/2406.09414

[22] Guy Yariv, Itai Gat, Lior Wolf, Yossi Adi and Idan Schwartz. AudioToken: Adaptation of Text-Conditioned Diffusion Models for Audio-to-Image Generation, Interspeech, 2023. https://arxiv.org/abs/2305.13050

[23] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman and Oliver Wang. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric, CVPR, 2018. https://arxiv.org/abs/1801.03924

[24] Jinxing Zhou, Jianyuan Wang, Jiayi Zhang, Weixuan Sun, Jing Zhang, Stan Birchfield, Dan Guo, Lingpeng Kong, Meng Wang and Yiran Zhong. Audio-Visual Segmentation, ECCV, 2022. https://arxiv.org/abs/2207.05042

[25] LAION-AI (Yusong Wu, Ke Chen et al.). CLAP: Contrastive Language-Audio Pretraining, code repository, 2022. https://github.com/LAION-AI/CLAP. Accessed on: 25th October, 2026.

[26] Patrick von Platen et al. (Hugging Face). Diffusers: State-of-the-art diffusion models, code repository, 2022. https://github.com/huggingface/diffusers. Accessed on: 25th October, 2026.

[27] Lihe Yang et al. Depth-Anything-V2, code repository, 2024. https://github.com/DepthAnything/Depth-Anything-V2. Accessed on: 25th October, 2026.

[28] Joseph Tung, Gene Chou et al. MegaScenes novel view synthesis code, code repository, 2024. https://github.com/MegaScenes/nvs. Accessed on: 25th October, 2026.